

24. Des arguments : le même geste, ailleurs

Une fonction qui dessine UNE boîte n'a servi qu'une fois ; avec des arguments, elle en dessine autant qu'on veut.

1. Ce qu'il faut comprendre

Les **paramètres** sont des variables qui reçoivent leur valeur **à l'appel** : `void boite(uint8_t x, uint8_t y, uint8_t largeur, uint8_t hauteur)`. Dans `boite(1, 1, 8, 4)`, x reçoit 1, y 1, largeur 8, hauteur 4 — **dans l'ordre**.

Les valeurs données à l'appel s'appellent les **arguments**. Ce peuvent être des nombres, des variables, des calculs : `boite(2 + 9, 1, ...)` marche aussi.

Dans la fonction, les paramètres sont des variables **locales** : ils n'existent que pendant l'appel, et un `x` ici n'a rien à voir avec un `x` de `main`.

Le corps mélange tout ce qu'on sait : une double boucle sur la surface, et une condition qui décide si la case est **au bord** (#) ou **dedans** (espace). C'est la grille de la leçon précédente — rendue **réutilisable**.

Quatre boîtes, **une** seule définition. Un menu, une fenêtre de dialogue, un inventaire : toutes les interfaces de jeu sont faites d'un geste répété avec d'autres arguments.

À toi : ajoute un cinquième argument, le caractère à écrire au milieu de la boîte.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **557 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
boite()	151 o	← main() — ta fonction
main()	115 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	58 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	80 o	← CopierTuiles — 5 lettres sur 43
"#"	1 o	← texte() — un texte gravé
" "	1 o	← texte() — un texte gravé
"MENU"	4 o	← texte() — un texte gravé

Lettres gravées : EMNU#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 3 71 o sur place, dans boite(), main() + AdresseCase (10 o), EcrireTexte (10 o)

```
ecran() x 2      21 o sur place, dans main() + AttendreVBlank (16 o)
image() x 1      3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
boite() x 4      76 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine quatre boîtes de tailles différentes avec UNE fonction.
//
// Ce qui est nouveau : les ARGUMENTS (ou « paramètres »).
// On donne des valeurs à la fonction entre les parenthèses ; elle fait le même
// geste, mais à un autre endroit, avec une autre taille.
// void boite(uint8_t x, uint8_t y, uint8_t largeur, uint8_t hauteur)
//   → boite(1, 1, 8, 4) : x = 1, y = 1, largeur = 8, hauteur = 4.

#include <texte>    // écrit un texte à l'écran
#include <ecran>   // éteint ou rallume l'écran
```

2. UNE FONCTION — BOITE()

```
void boite(uint8_t x, uint8_t y, uint8_t largeur, uint8_t hauteur) {
    // x, y = le coin en haut à gauche de la boîte.
    for (uint8_t l = 0; l < hauteur; l++) {      // chaque ligne l de la boîte
        for (uint8_t c = 0; c < largeur; c++) {    // chaque colonne c de la boîte
            uint8_t bord = l == 0 || l == hauteur - 1 || c == 0 || c == largeur - 1;
            // 1 si la case est sur le bord : première ligne,
            // dernière ligne (hauteur - 1), première colonne
            // ou dernière colonne (largeur - 1).
            if (bord) texte(x + c, y + l, "#"); // le bord : un #
            else texte(x + c, y + l, " ");      // l'intérieur : un espace (la boîte est vide)
        }
    }
}
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    boite(1, 1, 8, 4);                      // en (1, 1), 8 de large, 4 de haut
    boite(11, 1, 8, 6);                     // en (11, 1), 8 de large, 6 de haut
    boite(1, 8, 18, 3);                     // une longue boîte plate
    boite(6, 12, 8, 5);                     // la boîte du menu

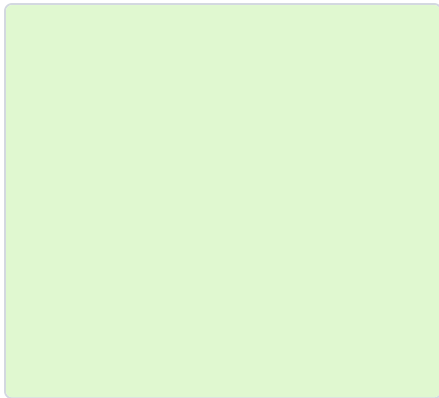
    texte(8, 14, "MENU");                   // Un mot au milieu de la dernière boîte.

    ecran(1);                               // On rallume.

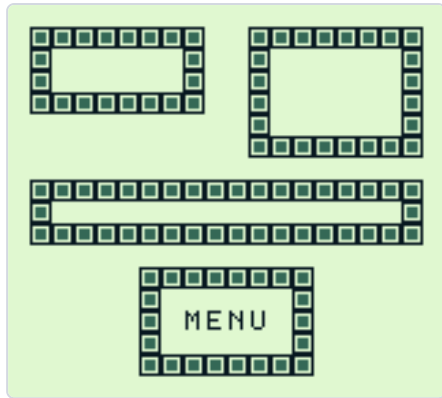
    while (true) {                           // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Quatre boîtes de tailles différentes, et « MENU » dans la dernière.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première boîte : 8 de large
- ✓ la deuxième : 6 de haut
- ✓ la troisième : 18 de large
- ✓ la dernière entoure MENU

← [23. Une fonction : donner un nom à un geste](#)

[sommaire](#)

[25. Une valeur de retour : la fonction qui répond](#) →